Shard Spiral: A Generic, User-Driven Visualization Platform for Algorithms and Data Structures

Dwarakesh Venkatesh

Abstract

Understanding algorithms and data structures is a persistent challenge in computing education because their behavior is dynamic and depends on many interacting pieces of state, and this challenge is sharpest for graphs and trees, which most computer science syllabi treat as the hardest structures to teach. Most existing visualization tools either replay a fixed animation of a canned example, require a developer to manually instrument their own code with visualization calls before anything can be drawn, or both, and nearly all of them are built and tuned for a desktop screen. We present Shard Spiral, a mobile-first, browser-based visualization platform built on a generic, zero-instrumentation tracer capable of visualizing arbitrary real code, with graphs and trees implemented first as the platform's proving ground precisely because they are the hardest case in the syllabus. The tracer pauses execution at algorithmic decision points so that the learner, not a script, resolves what happens next. Beyond this Manual Mode, Shard Spiral ships a free-form input parser, a customizable pseudo-random input generator capable of producing constraint-specific structures (such as acyclic graphs, strictly connected graphs, and strictly sorted arrays), a live state-variable panel with human-readable step explanations, a CodeMirror-based code editor with execution-synced line highlighting in which students can write and visualize their own algorithms, an Evaluation system to check their answers, and a live, in-canvas annotation system for instructor-led delivery. This paper situates these features against the engagement-level framework from algorithm-visualization research and compares Shard Spiral directly against six widely used reference tools: PythonTutor, VisuAlgo, Algorithm Visualizer, the University of San Francisco's Data Structure Visualizations, Pathfinding Visualizer, and GraphOnline. We argue, with supporting comparison tables, that Shard Spiral's combination of real execution, generic tracing, customizable constraint-based input generation, runtime learner control, result-based evaluation, live annotation, and genuinely mobile-capable design gives it concrete advantages that each reference tool is documented or observed as lacking, and we outline a concrete plan for the controlled classroom evaluation needed to confirm these advantages empirically.

Keywords

Algorithm Visualization, Data Structures, Interactive Learning, Graph Visualization, User-Driven Visualization, Constrained Random Input Generation, Pyodide, CodeMirror

1. Problem and Motivation

Algorithms and data structures are core components of undergraduate computer science curricula, yet students routinely find them difficult because their behavior is dynamic and multi-state: a single step of a traversal or shortest-path algorithm simultaneously changes a frontier, a visited set, and a table of distances. This difficulty is not uniform across structures. Graphs and trees are consistently the structures students and instructors find hardest, since their state is relational rather than linear and a single operation can touch many interconnected elements at once, unlike the more contained, sequential state of an array or a stack. Static instructional material-textbook figures, slides, and board-based tracing freezes a single instant of this process and leaves the learner to reconstruct the intervening motion by hand. This encourages memorization of a procedure rather than an intuitive model of why the procedure behaves the way it does.

Algorithm visualization (AV) has long been proposed as a remedy, making abstract computation visible over time [1, 2, 4]. Critically, however, the literature on AV effectiveness converges on a specific and often overlooked finding: visualizations that learners only watch produce little to no measurable learning benefit over not using a visualization at all. Benefit appears reliably only once learners move from watching to

acting-predicting outcomes, changing inputs, or constructing their own problem instances [2, 3]. Most deployed visualizers stop short of this threshold. A second, separate gap is physical rather than pedagogical: the most widely used tools in this space are explicitly designed, and in some cases explicitly documented, as desktop-only experiences, which excludes a growing share of learners who study primarily on a phone or tablet. A third gap is instructional rather than individual: almost none of these tools provide any way for an instructor to mark up a live visualization during a lecture, forcing classrooms to bolt a separate whiteboard application onto a screen share.

Shard Spiral is built to close all three gaps at once, and it is built as a generic visualization platform rather than a single-purpose one: its tracing engine is not specific to any one data structure and is designed to visualize arbitrary real code. Graphs and trees were chosen as the structures to implement first precisely because they are the hardest case in the syllabus; a platform capable of handling their relational, cyclic, and branching state generalizes readily to the more contained state of arrays, stacks, queues, and similar linear structures. It is a proprietary, independent project that runs real Python in the browser and pauses execution at genuine decision points, ships a customizable constraint-based input generator alongside free-form parsing, is built mobile-first from the ground up, and integrates a live annotation layer for classroom use. In the sections that follow we compare it against six separate, independently developed tools, each representative of a different design tradition in this space.

2. Related Tools

PythonTutor is a widely used, general-purpose code tracer that executes real code and draws a frame-by-frame memory diagram of variables, stack frames, and heap objects [5]. It is designed primarily for desktop and laptop screens, where its generic memory layout can become cluttered on smaller mobile viewports. Because it renders a low-level memory view rather than a domain-specific graph or tree canvas, a learner must translate pointer-and-box diagrams into a mental graph image themselves.

VisuAlgo is a long-established, professionally animated visualizer covering dozens of classic algorithms [6]. The platform was architected around desktop-class resolutions (1366x768 or larger), with only a reduced mobile lite version exposing a subset of desktop features introduced later. Its animations are hand-authored per algorithm, and the learner's role is largely limited to selecting an input and watching.

Algorithm Visualizer is an online platform built around the idea of visualizing algorithms from code. In practice, this requires the developer to manually instrument the algorithm with calls into language-specific tracer APIs (such as Array1DTracer or GraphTracer) before any visualization appears. The visualization is therefore only as complete as the manual instrumentation a human has already written into that specific algorithm; a new, un-instrumented algorithm draws nothing.

The University of San Francisco's Data Structure Visualizations (commonly known as the Galles visualizations) is one of the longest-standing academic tools in this space, covering a broad library of structures and algorithms through hand-built, simulated animations. No underlying user code is executed; the tool plays a scripted animation of the textbook algorithm's behavior, and input is limited to inserting individual values rather than constructing an arbitrary custom or constraint-specific graph or tree.

Pathfinding Visualizer is a popular grid-based tool focused narrowly on pathfinding algorithms such as Dijkstra's algorithm and A* on a two-dimensional maze-like grid. It provides no general graph construction, no code view, and no state panel; its scope is intentionally restricted to a single grid metaphor.

GraphOnline lets users construct an arbitrary custom graph and run classic graph algorithms (shortest path, spanning tree, and related operations) on it. Like the scripted tools above, execution is a deterministic, pre-programmed animation; no underlying source code is shown or run, and there is no point at which the learner's choice can redirect the algorithm's control flow.

3. Shard Spiral: System Overview

3.1 Real Execution, Not a Recorded Script

Shard Spiral runs genuine Python inside a Web Worker via Pyodide, a WebAssembly build of CPython. The source code shown beside the canvas is not illustrative; it is the code that is actually executing. This removes any possibility of drift between what the learner is told an algorithm does and what actually runs, a risk inherent to any hand-scripted animation such as those used by VisuAlgo [6], the USF visualizations, Pathfinding Visualizer, and GraphOnline.

3.2 A Generic, Zero-Instrumentation Tracer

Rather than special-casing variable names, or requiring the developer to manually call a tracer API as in Algorithm Visualizer, Shard Spiral's tracer uses sys.settrace together with Abstract Syntax Tree (AST) parsing to intercept language-level operations-indexing into a sequence, iterating over a graph's neighbors-so that new algorithms inherit canvas rendering and state tracking automatically, with no instrumentation code required.

3.3 CodeMirror-Based Editor

The code pane is built on CodeMirror, loaded directly with a focused set of plugins rather than a plain text area. This provides syntax highlighting, gutter line numbers, indent guides, and, most importantly, custom line-level highlighting driven by the tracer: as execution advances, the exact line currently running is highlighted in the gutter and body in sync with the canvas, establishing a direct visual link between a line of code and its effect on the data structure.

3.4 Manual Mode: Runtime Learner Control

The defining interactive capability is Manual Mode. Because the Python thread is paused synchronously by a custom exception, execution can stop mid-loop and ask the learner to resolve the next step-for example, which of two remaining neighbors a shortest-path algorithm should inspect next. The learner answers either by clicking a labeled control or by clicking the corresponding node or edge directly on the canvas, and the chosen branch is fed back into the running interpreter. This capability is absent from every reference tool discussed in Section 2: in each of them, the algorithm's control flow is never actually open to the learner.

3.5 Flexible, Format-Agnostic Input

Shard Spiral accepts a graph as an adjacency list, adjacency matrix, or edge list typed in a compact free-text shorthand, auto-validating and parsing it without requiring a rigid, pre-defined template. This provides a meaningfully lower-friction path from a learner's own homework graph to a working visualization of it than tools that rely mainly on code, fixed example sets, or single-value insertion.

3.6 Customizable Pseudo-Random Input Generator

Complementing free-form text parsing, Shard Spiral includes a customizable pseudo-random input generator that lets learners and instructors synthesize test cases tailored to specific structural invariants and edge cases. Rather than producing purely uniform, unconstrained random structures, users can parameterize structural properties directly-such as generating graphs guaranteed to have no cycles (DAGs/trees), graphs that are strictly connected, graphs with controlled edge density or weight ranges, or linear arrays that are strictly sorted, reverse-sorted, or nearly sorted. This allows students and instructors to rapidly stress-test algorithmic behavior across boundary conditions (for example, verifying cycle detection on acyclic versus cyclic instances, or observing traversal degradation on sparse versus dense connected components) without tedious manual input construction.

3.7 Time-Travel Debugging

Every traced state is retained, so a learner can step backward through an auto-run, or, uniquely, back out of a choice made in Manual Mode and try a different edge. This turns each decision point into a safe, repeatable hypothesis test: choose, observe the effect on the state table, and revise.

3.8 Live State Variables and Explanation Panel

A dedicated panel shows the current value of every relevant variable, pulled directly from the running code and updated in real time as it morphs, alongside a separate Explanation view that narrates every step of the algorithm in human-readable language rather than raw code or a static description.

3.9 Canvas Customization

The canvas background grid is itself configurable, with a selection of styles (dotted, squared, and others), in addition to light and dark themes that a learner or

instructor can choose to suit personal preference or presentation context.

3.10 Curriculum and Favorites Filtering

The explorer panel supports two complementary filtering views: a syllabus filter, through which an instructor can scope the visible algorithms and structures to what a given course or unit actually covers, and a favorites filter, through which an individual learner can pin the items they personally want quick access to. The syllabus list is currently configured through a static file, with instructor-side control planned as part of the cloud functionality described in Section 10.

4. Instructor and Assessment Features

4.1 Live In-Canvas Annotation

Shard Spiral includes a full annotation layer that can be used either on a running algorithm in real time or on a frozen canvas, letting an instructor draw freehand, draw predefined shapes, and type text directly over the live visualization rather than switching to a separate whiteboard application. The annotation tool supports undo and redo, a dedicated eraser, selection of existing strokes by drawing over them, a clear-all control, and export of the annotated canvas, making it suitable for both live lecture delivery and the creation of reusable, annotated reference material. No reference tool discussed in Section 2 offers an equivalent in-canvas annotation capability; instructors using those tools must overlay a separate screen-annotation application.

4.2 Open-Ended Implementation

To facilitate unconstrained algorithmic exploration, Shard Spiral provides a student-authored implementation environment. While the system supplies a standardized function signature (e.g., my_algorithm(adj_list, *args, **kwargs)) to ensure structural interoperability with the underlying generic tracer, the internal logic authored by the student is entirely arbitrary. This code is executed and visualized dynamically, serving as a purely formative sandbox.

4.3 Evaluation

Shard Spiral includes an Evaluation system that allows students to assess their understanding of an algorithm by submitting their answers to a given problem. Rather than evaluating the code or implementation used to arrive at a result, the system evaluates the correctness of the student's submitted answer against the expected outcome. Importantly, the evaluation accounts for problems that admit multiple valid answers. For example, in Breadth-First Search (BFS), multiple traversal orders may be valid depending on the order in which neighboring vertices are visited. Shard Spiral recognizes these alternative valid outcomes and accepts any answer that satisfies the algorithm's correctness requirements. Beyond simply indicating whether an answer is correct or incorrect, the Evaluation system provides explanatory feedback when an answer is incorrect, identifying why the submitted answer does not satisfy the expected algorithmic behavior. This helps students understand the reasoning behind their mistakes. By accepting alternative valid answers and providing explanations for incorrect ones, the Evaluation system offers a formative assessment mechanism that encourages students to evaluate their understanding and learn from their errors.

5. Why Visualization Must Be Interactive

The theoretical basis for this design is Naps et al.'s engagement taxonomy [1], which ranks learner interaction with a visualization across six levels: No Viewing, Viewing, Responding, Changing, Constructing, and Presenting, and predicts that learning outcomes improve as engagement rises through this hierarchy. Hundhausen et al.'s meta-study [2] found that how students interact with a visualization, not whether one exists, is what determines learning gains. Banerjee et al. [3] confirmed this directly in a classroom study: a Responding condition produced statistically significant gains in cognitive achievement over a matched Viewing condition using the identical visualization. Shaffer et al.'s survey of the field [4] likewise notes that most deployed AV tools remain at the Viewing level despite this evidence.

Table 1 maps each reference tool onto this taxonomy. Shard Spiral is the only one that reaches Changing through Manual Mode and customizable constraint-based input generation, and it additionally reaches Constructing where the learner authors the algorithm itself, and Presenting through live annotation, where an instructor narrates and marks up a run for an audience.

Table 1. Engagement level reached, by tool (Naps et al. taxonomy [1])

Tool	Engagement	Mechanism
PythonTutor [5]	Viewing	Steps fwd/back through a fixed trace
VisuAlgo [6]	Viewing	Pre-scripted animation; example switching
Algorithm Visualizer	Viewing	Runs pre-instrumented code; no runtime input
USF Data Struct.	Viewing	Scripted animation of textbook behavior
Pathfinding Vis.	Viewing	Scripted animation on a fixed grid
GraphOnline	Viewing	Scripted animation on a custom graph
Shard Spiral	Changing / Constructing / Presenting	Manual Mode, constrained random generator, Evaluation system, live annotation

6. Comprehensive Feature Comparison

Table 2 compares all seven tools across the dimensions most relevant to classroom use. A check mark (✓) indicates the capability is present and usable by an end user without extra engineering effort; a cross (✗) indicates it is absent or requires work outside the tool itself (such as writing tracer-instrumentation code).

Table 2. Feature comparison across seven visualization tools

Feature	PyTutor	VisuAlgo	AlgoVis.	USF	Pathfind.	GraphOnl.	Shard Spiral
Executes real code	✓	✗	✓	✗	✗	✗	✓
No manual instr. needed	✓	n/a	✗	n/a	n/a	n/a	✓
Domain-specific canvas	✗	✓	✓	✓	✓	✓	✓
Free-form/agnostic input	✗	✗	✗	✗	✗	✓	✓
Constrained random input	✗	△	✗	✗	△	✗	✓
Learner alters flow	✗	✗	✗	✗	✗	✗	✓
Evaluates submitted ans.	✗	✗	✗	✗	✗	✗	✓
Live in-canvas annotation	✗	✗	✗	✗	✗	✗	✓
Time-travel rewind	✓	△	△	△	△	△	✓
Step explanation panel	✗	△	✗	✗	✗	✗	✓
Responsive, mobile layout	✗	△	✗	✗	△	△	✓

✓ = present and usable by an end user; △ = partial or limited (e.g., VisuAlgo and Pathfinding Visualizer provide limited preset/random generation but lack general cross-structure constraint customization across arbitrary user-driven runs); ✗ = absent; n/a = not applicable, since Algorithm Visualizer's instrumentation requirement makes "no manual instrumentation needed" the relevant comparison only for tools that execute real code at all.

Two results stand out. First, Shard Spiral is the only tool in Table 2 that combines a domain-specific canvas with a fully generic, zero-instrumentation execution engine; VisuAlgo [6], the USF visualizations, Pathfinding Visualizer, and GraphOnline achieve the canvas only by paying a fixed per-algorithm hand-authoring cost, Algorithm Visualizer achieves real execution only by requiring manual tracer instrumentation, and PythonTutor [5] achieves real execution only by giving up the domain-specific canvas. Second, Shard Spiral is the only tool, among all seven, with rows checked for customizable constraint-based input generation alongside free-form parsing, runtime control flow, evaluation of submitted answers, and live annotation-capabilities that do not exist together anywhere else in this comparison.

7. A Mobile-Rich Interface as a Distinct Advantage

Mobile support is not a cosmetic detail in this comparison; it is a documented, structural gap in the two most widely used reference tools. PythonTutor [5] was architected specifically for desktop and laptop screens rather than mobile devices, resulting in a UI that becomes cramped and difficult to navigate on small viewports. Similarly, VisuAlgo [6] was designed around desktop resolutions (1366x768 or larger) and only later introduced a separate, reduced mobile lite mode exposing a subset of the desktop feature set. The remaining tools compared in Table 2 rely on precise mouse-driven interactions, such as drag-and-drop graph construction or click-and-drag grid painting, that are not touch-optimized and are cumbersome on a small screen.

Shard Spiral's interface was built the opposite way: a single responsive layout that stacks the canvas, code, and state panels on narrow screens and lays them side by side on wide ones, with every interaction, including Manual Mode's node and edge selection, the customizable random input generator, and the annotation layer's freehand drawing, usable by touch. This matters pedagogically as well as practically. Learners increasingly study in short sessions on a phone between classes, and a tool that degrades to a cluttered or feature-reduced experience on that device effectively excludes that study pattern; Shard Spiral's mobile-first design keeps the full feature set, including Manual Mode, the Evaluation system, and annotation, available on any screen size.

8. Summary of Advantages

Table 3 consolidates the concrete advantages argued for above into a single list, each one grounded either in a documented or observed limitation of the reference tools or in a capability unique to Shard Spiral among the seven tools compared.

Table 3. Shard Spiral advantages and their basis

Advantage	Basis for the Claim
Full feature set on mobile	PythonTutor [5] and VisuAlgo [6] rely on desktop-oriented or reduced mobile experiences, and the remaining tools depend on precise mouse interactions; Shard Spiral ships one responsive, touch-capable UI.
Runtime learner control	Manual Mode pauses execution synchronously so a learner's choice directs the active branch; no reference tool exposes runtime control flow to the learner.
Zero-instrumentation tracer	Algorithm Visualizer requires manual tracer API calls; VisuAlgo [6], USF, Pathfinding Visualizer, and GraphOnline hand-author each animation; Shard Spiral's AST/settrace engine needs neither.
Domain-specific canvas with real execution	PythonTutor [5] executes real code but shows only a generic memory view; the remaining tools provide a structural canvas without executing unmodified user code.

Format-agnostic input	Shard Spiral infers adjacency-list, adjacency-matrix, or edge-list shorthand automatically, without requiring a rigid template or manual setup code.
Customizable pseudo-random input generator	Generates constraint-specific test structures on demand-including acyclic graphs (DAGs/trees), strictly connected graphs, and strictly sorted arrays-to test algorithmic edge cases.
Evaluation of submitted answers	Shard Spiral evaluates the correctness of a student's submitted answer, accounts for multiple valid outcomes, and provides explanatory feedback, rather than evaluating code implementation.
Live in-canvas annotation	Freehand drawing, geometric shapes, text, and stroke erasing operate directly over a live or paused visualization without external screen-overlay software.
Reversible, branch-aware rewinding	Retains full state history so learners can step backward out of a Manual Mode decision and test alternative branches as repeatable hypothesis tests.

9. Contribution and Significance

Taken together, Shard Spiral's contribution rests on four pillars: (1) a single generic, zero-instrumentation tracer that removes the per-algorithm engineering cost paid by every scripted or manually instrumented reference tool; (2) a dual-path input subsystem combining a format-agnostic shorthand parser with a customizable pseudo-random generator capable of enforcing structural invariants (such as acyclicity, strict connectivity, and strict array ordering), lowering the friction between a learner's problem instance and a working visualization of it; (3) Manual Mode which moves the tool through the Changing and Constructing levels of the engagement taxonomy [1], levels that none of the six reference tools reach; and (4) a live annotation layer paired with a genuinely mobile-rich interface, which extends the tool's reach to the Presenting level and to any device a learner or instructor has at hand.

From a computing-education standpoint, the Evaluation system in particular opens an assessment design that none of the six reference tools support: an instructor can assign a traversal or shortest-path problem, let the student submit their answer, and receive an automatically evaluated result that accounts for multiple valid outcomes and provides explanatory feedback, rather than evaluating the implementation code used to arrive at the result. Manual Mode and the constrained random generator complement this by letting an instructor generate targeted boundary cases-such as strictly connected graphs or acyclic topologies-and ask a student to resolve every tie-break in an existing algorithm by hand and explain why a particular choice does or does not still yield a correct result, turning the visualizer itself into an assessment instrument rather than only a demonstration aid.

10. Future Work

Controlled classroom evaluation. The next stage is a between-subjects study with undergraduate data-structures students, comparing a Shard Spiral Manual Mode and Evaluation system condition against a Viewing-only condition on the same algorithms, following the design used by Banerjee et al. [3], measuring conceptual understanding, time-on-task, and self-reported engagement.

Cloud-based session sharing. A single shareable link that reproduces an exact session, including its generated or parsed graph or tree, its trace position, and any annotations, is in development, so an instructor can reproduce a specific moment of a lecture for students to revisit asynchronously.

Cloud-based persistence. Saving of learner-created structures and instructor annotations to the cloud, rather than only within a session, is in development, alongside instructor-side control of the syllabus filter described in Section 3.10, which is currently configured through a static file.

Breadth beyond graphs and trees. The underlying tracer, constrained input generator, and canvas architecture are not specific to any one data structure; graphs and trees were implemented first because they are the hardest case in the syllabus, and the platform already covers a working set of traversal and shortest-path algorithms on them, with others in active

development. Extending full canvas rendering support to arrays, stacks, queues, linked lists, and heaps-alongside strictly sorted, reverse-sorted, and nearly sorted array generators-is the natural next step, and requires comparatively less engineering effort given their simpler, linear state.

Visualization support for additional languages. Implementations already exist in languages beyond Python, and code and notes in those languages can be viewed, copied, and downloaded; extending live browser-based visualization itself, currently Python-only, to those languages is a planned direction.

11. Conclusion

Shard Spiral advances the state of algorithm visualization along the axis that algorithm-visualization research identifies as decisive: how much control the learner has over the visualization [1, 2, 3], not merely how polished an animation looks, and on how many of the learner's own devices that control is actually available. By running real Python through a generic, zero-instrumentation tracer, supporting both format-agnostic input and customizable constraint-based random generation, and pausing execution at genuine decision points, Shard Spiral reaches the Changing level of Naps et al.'s engagement taxonomy [1], while its Evaluation system and live annotation layer extend it further into Constructing and Presenting-levels that none of the six reference tools compared here reach in their current deployed forms. By building its interface mobile-first rather than retrofitting it afterward, Shard Spiral also avoids the desktop-only or reduced-mobile limitations of existing tools [5, 6]. The comparisons in Tables 1 through 3 make these advantages concrete rather than impressionistic. The work that remains, detailed in Section 10, is not a change of direction but an extension of this same architecture to more algorithms, more structures, more languages, and, most importantly, a controlled classroom study that converts the comparative case made here into measured empirical evidence.

References

[1] T. L. Naps, G. Rößling, V. Almstrum, W. Dann, R. Fleischer, C. Hundhausen, A. Korhonen, L. Malmi, M. McNally, S. Rodger, and J. Á. Velázquez-Iturbide. 2002. Exploring the role of visualization and engagement in computer science education. ACM SIGCSE Bulletin 35, 2 (2002), 131–152.

[2] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko. 2002. A meta-study of algorithm visualization effectiveness. Journal of Visual Languages & Computing 13, 3 (2002), 259–290.

[3] G. Banerjee, S. Murthy, and S. Iyer. 2015. Effect of active learning using program visualization in technology-constrained college classrooms. Research and Practice in Technology Enhanced Learning 10, 1, Article 15 (2015), 1–25.

[4] C. A. Shaffer, M. L. Cooper, A. J. D. Alon, M. Akbar, M. Stewart, S. Ponce, and S. H. Edwards. 2010. Algorithm visualization: The state of the field. ACM Transactions on Computing Education (TOCE) 10, 3, Article 9 (Aug. 2010), 1–22.

[5] P. J. Guo. 2013. Online Python Tutor: Embeddable web-based program visualization for CS education. In Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE '13). ACM, New York, NY, USA, 579–584.

[6] S. Halim. 2015. VisuAlgo – Visualising data structures and algorithms through animation. Olympiads in Informatics 9 (2015), 243–245.